

紀念品

Amaru 正在外國商店購買紀念品。那裡有 N 種**類型**的紀念品。商店裡每個類型的紀念品都有無限多個可供選擇。

每種紀念品都有固定的價格。也就是說，一件類型為 i 的紀念品 ($0 \leq i < N$) 的價格為 $P[i]$ 枚硬幣，其中 $P[i]$ 是正整數。

Amaru 知道紀念品種類是依照價格從高到低排列的，且紀念品的價格各不相同。具體來說， $P[0] > P[1] > \dots > P[N-1] > 0$ 。再者，他可以知道 $P[0]$ 的價值。不幸的是，Amaru 沒有關於其他任何紀念品價格的資訊。

為了購買一些紀念品，Amaru 會與賣家進行一些交易。

每筆交易均包含以下步驟：

1. Amaru 向賣家交付一些（正數）硬幣。
2. 賣家把這些硬幣堆放在後面房間的桌上，而 Amaru 是看不見的。
3. 賣家依次序逐一考慮每種紀念品類型 $0, 1, \dots, N-1$ 。每筆交易中每種類型僅被考慮為一次。
 - 當考慮到紀念品類型 i 時，如果目前硬幣堆中的數量至少為 $P[i]$ ，那麼
 - 賣家從堆中取出 $P[i]$ 個硬幣，並且
 - 賣家在桌上放置一件類型為 i 的紀念品。
4. 賣家把桌上剩下的所有硬幣和紀念品都給了 Amaru。

請注意，交易開始前桌上不會有硬幣或紀念品的。

您的任務是指示 Amaru 執行一些交易，以便：

- 每次交易他都會購買到**至少一件**紀念品，並且
- 總的來說，最後他會**剛好**買了類型 i 的紀念品 i 件，對每個 i ， $0 \leq i < N$ 。請注意，這意味著 Amaru 不應該購買類型 0 的紀念品。

Amaru 無需把交易次數最少化，並且擁有無限量的硬幣可供使用。

實作細節

您應該編寫以下的子程序。

```
void buy_souvenirs(int N, long long P0)
```

- N ：紀念品種類的數量。

- P_0 : $P[0]$ 的值。
- 對於每個測試樣例，這個子程序只會被呼叫一次。

上述程序可以呼叫以下子程序以指示 Amaru 執行交易：

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : Amaru 交給賣家的硬幣數量。
- 這個子程序會返回兩個元素。第一個元素是數組 L ，包含已購買的紀念品類型（按升序排列）。第二個元素是整數 R ，交易後返還給 Amaru 的硬幣數量。
- 要求 $P[0] > M \geq P[N-1]$ 。條件 $P[0] > M$ 確保 Amaru 不會購買任何類型為 0 的紀念品，而 $M \geq P[N-1]$ 確保 Amaru 至少購買一件紀念品。如果不滿足這些條件，您的解決方案將立即收到 Output isn't correct: Invalid argument 的最後結果。請注意，與 $P[0]$ 相反，輸入中未提供 $P[N-1]$ 的值。
- 每個測試樣例中該子程序最多可被呼叫 5000 次。

樣例評分程式的行為是**不自適應**的。這意味著在呼叫 `buy_souvenirs` 之前價格序列 P 是固定的。

約束

- $2 \leq N \leq 100$
- 對於每個 i ， $1 \leq P[i] \leq 10^{15}$ 且 $0 \leq i < N$ 。
- 對於每個 i ， $P[i] > P[i+1]$ 且 $0 \leq i < N-1$ 。

子任務

子任務	分數	附加約束
1	4	$N = 2$
2	3	對每個 i ， $P[i] = N - i$ 且 $0 \leq i < N$ 。
3	14	對每個 i ， $P[i] \leq P[i+1] + 2$ 使得 $0 \leq i < N-1$ 。
4	18	$N = 3$
5	28	對每個 $P[i+1] + P[i+2] \leq P[i]$ ， i $0 \leq i < N-2$ 。 對每個 $P[i] \leq 2 \cdot P[i+1]$ ， i $0 \leq i < N-1$ 。
6	33	沒有額外的限制。

例子

考慮以下調用。

```
buy_souvenirs(3, 4)
```

有 $N = 3$ 種紀念品， $P[0] = 4$ 。請注意，價格 P 只有三種可能序列： $[4, 3, 2]$ 、 $[4, 3, 1]$ 和 $[4, 2, 1]$ 。

假設 `buy_souvenirs` 呼叫 `transaction(2)`。假設呼叫返回 $([2], 1)$ ，這意味著 Amaru 購買了一件類型為 2 的紀念品，賣家退還給他 1 硬幣。請注意，這使我們能夠推斷出 $P = [4, 3, 1]$ ，因為：

- 對於 $P = [4, 3, 2]$ ，`transaction(2)` 將會回傳 $([2], 0)$ 。
- 對於 $P = [4, 2, 1]$ ，`transaction(2)` 將會回傳 $([1], 0)$ 。

然後 `buy_souvenirs` 可以呼叫 `transaction(3)`，返回 $([1], 0)$ ，這意味著 Amaru 購買了一件類型為 1 的紀念品，而賣家退還給他 0 硬幣。到目前為止，總共他購買了一份類型為 1 的紀念品和一份類型為 2 的紀念品。

最後，`buy_souvenirs` 可以呼叫 `transaction(1)`，返回 $([2], 0)$ ，這意味著 Amaru 購買了一份類型為 2 的紀念品。請注意，我們也可以在這裡使用 `transaction(2)`。至此，總計 Amaru 擁有一件類型為 1 的紀念品和兩件類型為 2 的紀念品，正如所要求的。

樣例評分程式

輸入格式：

```
N
P[0] P[1] ... P[N-1]
```

輸出格式：

```
Q[0] Q[1] ... Q[N-1]
```

這裡 $Q[i]$ 是總共購買的 i 類紀念品的數量，對每 i 使得 $0 \leq i < N$ 。